

Informe 1: PSD DE SEÑALES ALEATORIAS (GNURADIO)

Carlos David Cuadros Sanabria- 2201524
Fabian Amador Ballesteros- 2204215

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones
Universidad Industrial de Santander
31 Agosto de 2025

1. Abstract

This case study presents the development of two practical exercises oriented to the use of GNU Radio in digital communications applications. Several signal processing blocks were complemented, such as an accumulator, a differential block, and a time-average block. In the first practical exercise, students implemented their own algorithms in Python blocks to perform operations such as accumulation, differentiation, and statistical calculations; in addition to exploring the use of the accumulator as a simple low-pass filter. The second practical exercise focused on the characterization of binary signals and white noise, analyzing their properties in the time and frequency domains, as well as the implementation of digital modulation schemes (OOK, BPSK, ASK). To this end, digital communication concepts were reinforced through the generation, manipulation, and analysis of discrete signals, applying theory to practice.

2. Keywords

GNU Radio, digital communications, signal processing, Python blocks, accumulator, differentiation, low-pass filter, statistical analysis, binary signals, white noise, time-domain analysis, frequency-domain analysis, digital modulation, OOK, BPSK, ASK, discrete signals.

3. Resumen

En este caso práctico, presenta el desarrollo de dos prácticas orientadas al uso de GNU Radio en aplicaciones de comunicaciones digitales. Se complementaron varios bloques de procesamiento de señales, como un acumulador, un bloque diferencial y un bloque de promedio temporal. En la primera práctica, los estudiantes imple-

mentaron algoritmos propios en bloques de Python para realizar operaciones como acumulación, diferenciación y cálculo estadístico; además de explorar el uso del acumulador como un filtro paso bajo simple. La segunda práctica se enfocó en la caracterización de señales binarias y ruido blanco, analizando sus propiedades en el dominio del tiempo y la frecuencia, así como la implementación de esquemas de modulación digital (OOK, BPSK, ASK). Para ello, se reforzaron los conceptos de comunicación digital mediante la generación, manipulación y análisis de señales discretas, aplicando la teoría a la práctica.

4. Metodología

Inicialmente, se crea una rama para el desarrollo de la práctica desde el repositorio, denominada Práctica 1, la cual debe ser compartida con los demás integrantes del grupo. Se implementaron dos bloques de Python en el entorno de GNU-Radio: un acumulador y un diferenciador, con ayuda del libro guía propuesto en la práctica, en la sección 1.2.0.1 [1], se obtuvieron los códigos guías para implementarlos en los respectivos bloques.

4.1. Bloque acumulador

El bloque de Python denominado Accumulator se programó para sumar de manera consecutiva las muestras de la señal de entrada según la ecuación: Para evitar que la salida permaneciera cercana a ce-

$$y[n] = y[n-1] + x[n]$$

Fig. 1: Ecuacion Bloque acumulador

ro con una señal cuadrada simétrica, se aplicó un desplazamiento de -0.5 a la fuente. Este offset impide la cancelación entre valores positivos y negativos, lo que permite apreciar de manera clara la evo-

lución acumulada. Finalmente, se conectó la salida del bloque y la señal de entrada a un visualizador en tiempo real, a fin de comparar la forma de onda original y la forma procesada.

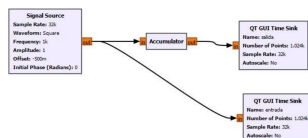


Fig. 2: Diagrama de flujo en GNU Radio que muestra un bloque acumulador

4.2. Bloque diferenciador

El bloque diferenciador es aquel que realiza la operación de la diferenciación, es decir, calcula la derivada con respecto al tiempo, de manera matemática se puede ver de la siguiente manera. Nuevamente

$$y[n] = x[n] + x[n - 1]$$

Fig. 3: Ecuacion Bloque Diferenciador

se emplea un bloque de Python en GNURadio para implementar un algoritmo que me haga esta operación, en donde se le ingresa una señal con el fin de obtener su derivada.

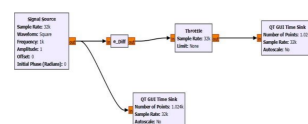


Fig. 4: Implementación del bloque diferenciador

A continuación, se debe crear un tercer bloque para mostrar las estadísticas de una señal. Este bloque, al igual que los anteriores, se implementa utilizando el bloque 'Python block' en GNU Radio, donde se realiza con base al libro guía sección 1.2.0.1 [1]. A partir de esta implementación se debe realizar una aplicación, en este caso disminuir la afectación del ruido Gaussiano en la estadística, para ello se divide la señal en 10 ventanas de tamaño 4096 muestras y

se decide utilizar funciones propias de Python para intentar comparar con las sugeridas por el libro.

4.3. Bloque promedio del tiempo

El bloque desarrollado, llamado Promedios de tiempos, fue definido como un bloque de sincronización (gr.sync block). Se inicializó con variables acumulativas para mantener un seguimiento del cálculo en tiempo real, evitando pérdida de información en flujos continuos de datos.

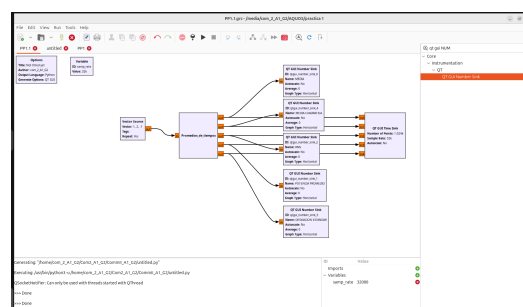


Fig. 5: Diagrama de flujo en GNU Radio que muestra las estadísticas de una señal.

Los siguientes procedimientos fueron implementados dentro del método work

4.4. método work

- **Media:** Se calcula acumulando la sumatoria de la señal y dividiéndola por la cantidad total de muestras recibidas.
- **Media cuadrática:** Similar a la media, pero utilizando los valores de la señal elevados al cuadrado.
- **Valor RMS:** Se obtiene tomando la raíz cuadrada de la media cuadrática.
- **Potencia promedio:** Se calcula elevando al cuadrado el valor RMS.
- **Desviación estándar:** Se obtiene mediante la sumatoria acumulada de la diferencia al cuadrado entre la señal y su media, dividiendo por el total de muestras y luego extrayendo la raíz cuadrada.

A partir de esta implementación se debe realizar una aplicación, en este caso disminuir la afectación del ruido Gaussiano en la estadística, para ello se divide la señal en 10 ventanas de tamaño 4096 muestras y

se decide utilizar funciones propias de Python para intentar comparar con las sugeridas por el libro.

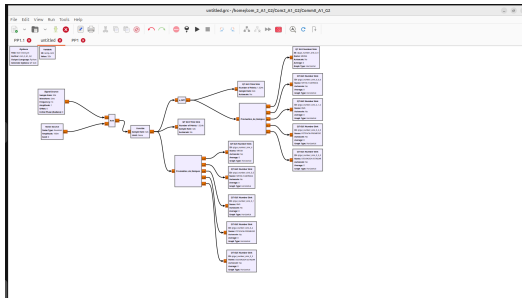


Fig. 6: Implementación de la aplicación de reducción de ruido Gaussiano con las modificaciones realizadas.

Resultados y Análisis del Flujo en GNU Radio

En la primera práctica (Fig. 5) se implementó un diagrama de flujo en GNU Radio utilizando una **fente de vector** con los valores $[1, 2, -1]$ repetidos continuamente. Estos datos fueron procesados mediante un bloque personalizado llamado **Promedios_de_tiempos**, encargado de calcular métricas estadísticas como media, desviación estándar, RMS, potencia promedio y media cuadrática. Las salidas se visualizaron en bloques QT GUI Number Sink y QT GUI Time Sink.

Los resultados obtenidos fueron los siguientes:

- **Media:** 0,667372
Este valor corresponde al promedio aritmético del vector, que teóricamente es:

$$\text{Media} = \frac{1 + 2 + (-1)}{3} = 0,6667$$

El valor obtenido coincide con el esperado.

- **Desviación estándar:** 1,247195
Calculada sobre el mismo vector, representa la dispersión de los datos alrededor de la media. El resultado coincide con el valor teórico.
- **RMS (Root Mean Square):** 1,414500
El valor RMS calculado coincide con el teórico, ya que:

$$\text{RMS} = \sqrt{\frac{1^2 + 2^2 + (-1)^2}{3}} = \sqrt{2} \approx 1,414$$

- **Potencia promedio:** 2,000809
Este valor corresponde a la media de los cuadrados de la señal, lo cual es consistente con el cálculo:

$$\frac{1^2 + 2^2 + (-1)^2}{3} = 2$$

- **Media cuadrática:** 2,000809
Presenta el mismo resultado que la potencia promedio, lo cual indica que ambos cálculos fueron implementados de manera similar.

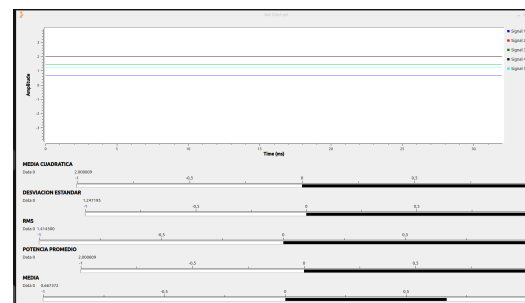


Fig. 7: RESULTADOS 1

Estos resultados validan el correcto funcionamiento del bloque **Promedios_de_tiempos** para un conjunto discreto de datos.

En la segunda práctica (Fig. 6) se utilizó una fuente de señal senoidal combinada con ruido gaussiano, sumadas mediante un bloque **Add**, para luego visualizar y analizar sus características estadísticas. Además, se aplicó un bloque de diferenciación para evaluar el comportamiento de la señal derivada. Los resultados obtenidos fueron:

- **Media de la señal original:** 0,115238
El valor es cercano a cero, lo cual es coherente con la naturaleza de la onda seno.
- **Desviación estándar:**
 - Señal original: 0,714066 → acorde a una señal senoidal con ruido de baja amplitud.
 - Señal derivada: 7,416229 → muy elevada debido a que la operación de derivación amplifica el ruido.
- **Potencia promedio:**
 - Señal original: 0,509893 → valor consistente para una onda seno con amplitud unitaria y ruido bajo.

- Señal derivada: 55,010925 $\rightarrow$ extremadamente alta, consecuencia directa de la amplificación del ruido al aplicar la derivada.

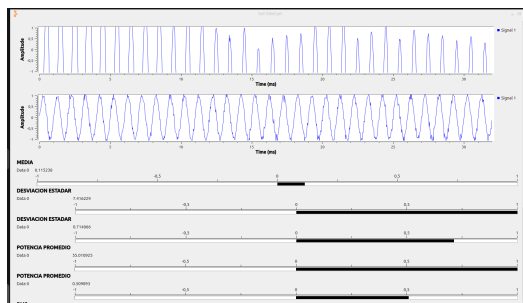


Fig. 8: RESULTADOS 2

Los resultados muestran que la señal original mantiene valores estadísticos dentro de lo esperado, mientras que la señal diferenciada presenta incrementos significativos en su potencia y desviación estándar debido al efecto del operador de derivación sobre el ruido.

5. Conclusiones

- Las pequeñas modificaciones en el código, como la corrección de la indentación y el orden de las líneas, fueron esenciales para el correcto funcionamiento de los bloques. Durante la implementación del bloque acumulador, se observó que en señales con media no nula, los valores de salida tienden a un desbordamiento en lugar de a infinito, lo que resalta la importancia de considerar las limitaciones físicas del hardware. La personalización del bloque en GNU Radio, ajustando el offset, no solo permitió visualizar correctamente la evolución de la señal, sino que también demostró la necesidad de adaptar las herramientas a las características específicas de cada señal para obtener resultados precisos.
- Referente al desarrollo de los códigos propuestos se le hicieron pequeñas modificaciones como arreglar la indentación, cambiar unas comillas y corregir el orden de unas líneas de código para su correcto funcionamiento. En el caso del bloque acumulador y diferenciador si tenemos una señal que su media no es cero, y sus valores están sobre el eje positivo los valores de salida de los bloques tienden a infinito, pero en la realidad de la implementación llegan a un punto donde se desbordan y reinicia la operación como ocurría en sistemas digitales.

Referencias

- [1] H. O. Boada and O. M. R. Torres, *Comunicaciones Digitales basadas en radio definida por software*, 1a ed. Bucaramanga, Colombia: Editorial UIS, 2019.
- [2] S.-M. M. Yang, *Review of Signals and Systems: Signals and Systems, and of Probability and Random Process*. Cham: Springer International Publishing, 2020. [Online].

https://github.com/Carlos-Cuadros240/CommIIA1_G2